Code Review

- Introduction
- Expectations
- Process
- Types of Code Review
  - Proof of Concept (POC) Reviews
  - Work in Progress (WIP) Reviews
  - Ready To Merge Reviews
- Expectations for Code Review
  - Code Authors
  - Code Reviewers
- Code Review Like a Human

- Examples of High Quality Pull Requests

Introduction

All code at Braze must be reviewed by a peer. Code review is meant to give you an opportunity to discuss your work and understand it more deeply. Review discussions are one of the best opportunities to learn and to share ideas; we should strive to make them as useful as possible.

Expectations

At Braze, code reviews are an integral part of our ability to ship high quality, performant code. As a result, providing quality, in-depth code reviews is an integral part of expectations for the role, just as important as writing good code.

Code review expectations here may differ from other organizations, so please work with your team and manager to ensure you're on the same page here!

Process

In general, the process for a code review is:

1. The author creates a pull request on Github that encapsulates your changes.
2. The author requests review for an individual using the `Reviewers -> RequestaReview` functionality on the pull request UI on Github. Note: this will ping the reviewer in slack, and will be visible in `#pull-review-requests`.
3. The reviewer uses Github's commenting tools to leave comments on the author's pull request.
4. The author responds to each comment by either making a change or starting a discussion.

When selecting who to tag as a reviewer, consider who is likely to have the most context on your work. Often this will be a tech lead, mentor, or manager, but it could also be a peer who you know has worked on a similar project or who is familiar with the part of the codebase that you have modified.

Tagging multiple reviewers is sometimes appropriate for larger or more complicated projects. Additionally, *if you tag Jon Hyman as a reviewer, you must tag a second reviewer*. This helps reduce the extent to which "key man risk" accumulates around the CTO. It will also improve your chances of getting a timely review, as Jon is very busy.

Types of Code Review

Code reviews come in three flavors:

- a “proof of concept” review
- a “work in progress” review
- a “ready to merge” review

Proof of Concept (POC) Reviews

A POC is a scaled down or rough version of a larger project, built in order to demonstrate the viability of a particular approach. Code review on a POC can be a good way to get feedback on a technical approach to a problem before making a large time investment coding a production-ready feature. You can send a pull request marked as `DRAFT` and explicitly titled with `POC` or `DO NOT MERGE - POC`. This will ensure the reviewer understands that they are dealing with a POC and not a ready to merge feature.

Work in Progress (WIP) Reviews

A WIP is code that is not ready to merge, but that you want a peer reviewer to be aware of. You can send a pull request marked as `DRAFT` and explicitly titled with `WIP` or `DO NOT MERGE - WIP`. This will ensure the reviewer understands that they are dealing with a WIP and not a ready to merge feature.

Ready To Merge Reviews

When you believe a project is completed and ready to be merged into `develop`, you must initiate a code review prior to merging. This is the only kind of code review that is mandatory.

Expectations for Code Review

Code Authors

Before initiating a code review:

- Thoroughly read over your changes before tagging someone else for review.
- For large projects, have a release plan and consider breaking a project into a few smaller pull requests. This helps de-risk projects and also makes your code easier to review.
- For “Ready To Merge” reviews, ensure your code is finalized, clean, and bug free. Pull requests should represent your best work.

When initiating a code review:

- Your review should use the pull request template (new pull requests for `Appboy/platform` default to this template).
- Explicitly state what feedback you would like from the review and also provide context so a reviewer can quickly get up to speed on your project.

During code review:

- Answer / acknowledge every comment left by the reviewer.
- If you need clarification on a comment or you have a question, seriously consider just talking to the person or asking via slack. It’s rarely efficient to pose clarification questions in the pull request and it can be a frustrating to go back and forth clarifying.
- Ultimately the responsibility for getting a code review done is on you. If you have requested code review but haven’t received a response from your reviewer, consider escalating your messaging (e.g. from a ping, to a ping with a message about urgency, to slack DM/conversation, etc.)

- Focus on code quality and functional correctness. Don't leave a comment merely because you would have coded something differently - leave a comment only when you perceive a material functional or maintenance impact.
- If you encounter systematic issues, make a single comment (i.e., this needs to be cleaned up) and send back to the reviewee. There's no need to enumerate every case.
- Please review the specs in addition to the main code. Broken specs are still broken code, and should be fixed before merge. Even if a broken spec is testing working code, the broken tests can hide bugs that get introduced later.
- It is your responsibility to explicitly approve a pull request when you think it is ready to merge. A corollary to this is that you must understand how the code you are reviewing works before you approve a pull request.
- Please review the specs, in addition to the code. One suggestion is that you read the tests first in order to get a sense for what the pull contains.

Code Review Like a Human

It's important to remember that, whether you are the author or the reviewer, there is a human on the other side. Check out these articles on how to participate in the code review process in a helpful and human way.

- As an author
  - https://mtlynch.io/code-review-love/
- As a reviewer
  - https://mtlynch.io/human-code-reviews-1/
  - https://mtlynch.io/human-code-reviews-2/

Below are excerpts from those articles that were most meaningful to me (Preston):

- As the author:
  - Answer questions with the code itself
    - When your reviewer expresses confusion about how the code works, the solution isn't to explain it to that one person. You need to explain it to everyone.
    - The best way to answer someone's question is to refactor the code and eliminate the confusion.
  - Separate functional and non-functional changes
    - If a piece of code requires refactoring and behavioral changes, it should happen in two to three changelists:
      - 1. Add tests to exercise the existing behavior (if they're not already there).
      - 2. Refactor the production code while holding the test code constant.
      - 3. Change behavior in the production code and update the tests to match.
  - Respond graciously to critiques
    - Treat your reviewer's notes as an objective discussion about the code, not your personal worth as a human. Responding defensively will only make things worse.
    - Surprisingly, it's a good sign when your reviewer spots subtle flaws in your code. It indicates that you're packaging your changelists well.
  - Be patient when your reviewer is wrong
    - Even when your reviewer is mistaken, that's still a red flag. If they misread it, will others make the same mistake?Artfully solicit missing information

        - What changes would be helpful?
            - I love this response because it signals a lack of defensiveness and openness to criticism. Whenever a reviewer gives me unclear feedback, I always respond with some variation
- As a reviewer:
    - Start high level and work your way down
        - The more notes you write in a given review round, the more you risk making the author feel overwhelmed.
        - Your low-level notes might become moot once the author integrates your high-level notes. By deferring them to a later round, you save yourself the nontrivial work of writing carefully-worded comments calling out the issues, and you spare the author from processing unnecessary notes.
    - Be generous with code example
        - If you lighten the author's load by writing out some of the changes you're suggesting, you demonstrate that you are generous with your time as a reviewer.
        - Reserve this technique for clear, uncontroversial improvements.
    - Never say "you"
        - The decisions you reach in a review should be based on what makes the code better rather than who came up with the idea.
        - Word your feedback in a way that minimizes the risk of raising your teammate's defenses. Be clear that you're critiquing the code, not the coder.
    - Frame feedback as requests, not commands
        - Code reviews require more tact and care than usual communication because there's a high risk of derailing the discussion into a personal argument.
        - Err on the side of being annoyingly gentle in your feedback. Frame your notes as requests or suggestions, not commands.
        - People like to feel in control of their own work. Making a request of the author gives them a sense of autonomy.
        - Requests also make it easier for the author to push back politely. Maybe they have a good reason for their choice.
    - Offer sincere praise
        - Most reviewers focus only on what's wrong with the code, but reviews are a valuable opportunity to reinforce positive behaviors.
        - Sincere compliments ease this tension by demonstrating that you are their supportive teammate and not the cruel gatekeeper.

Examples of High Quality Pull Requests

https://github.com/Appboy/platform/pull/33576 by @Singwai Chan